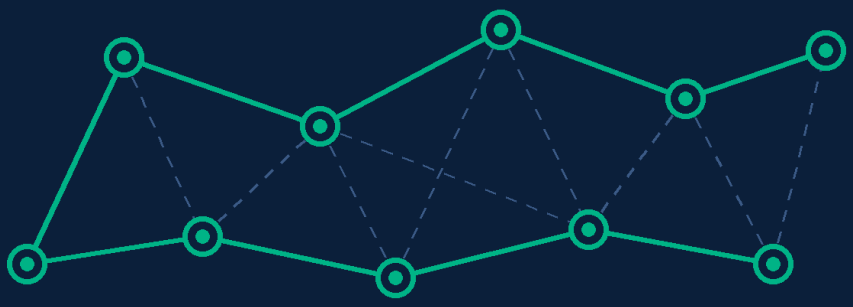
**NED University of Engineering and Technology**
Department of Computer Science and Information Technology

PROJECT PROPOSAL

# Smart EV Charging Station Planning and Energy Allocation System

## for Urban Pakistan



COURSE

**Design and Analysis of Algorithms · CT-363**

COURSE INSTRUCTOR

**Miss Tooba Shaikh**

GROUP MEMBERS

**Sarim Khan**
Roll No. CT-24266

**Muhammad Saraib**
Roll No. CT-24255

**Danish Meraj**
Roll No. CT-24268

**Muhammad Maaz Ali**
Roll No. CT-24265

## 01 Abstract

EV adoption in Pakistan needs charging infrastructure that is both economically connected and efficiently used. This prototype combines a **Greedy** algorithm (Kruskal's MST/MSF with Union-Find) that plans a minimum-cost connection network over candidate sites with an exact **Dynamic Programming** algorithm (0/1 multiple knapsack) that allocates limited station energy to competing EV requests. The stages are linked by an **eligibility rule**: an EV can only use an active station in the same connected component. It handles unavailable roads, repairs the network after three road failures, and is validated against full recomputation, brute force, edge cases and sensitivity experiments. All data is synthetic.

## 02 Problem Statement and Objectives

Given candidate sites, road links (some unavailable), a few active stations with finite energy and competing EV requests, find (a) a minimum-cost connection network and (b) an assignment of requests to stations that maximizes served benefit within every station's capacity. Motivated by Pakistan's New Energy Vehicles Policy 2025–30.

| Real-world decision | Algorithmic problem | Paradigm |
|---|---|---|
| **Connect sites at minimum cost** | Minimum Spanning Tree / Forest: Kruskal + Union-Find (DSU) | Greedy |
| **Allocate limited station energy to EVs** | 0/1 Multiple Knapsack with assignment reconstruction | Dynamic Programming |

**Objectives:** (1) model both problems formally; (2) argue correctness; (3) derive and measure time/space complexity; (4) repair the network after three road failures and check against full Kruskal; (5) validate DP against brute force; (6) run edge-case and sensitivity experiments; (7) present results on a dashboard.

## 03 Scope and Case Study

| Included | Not included (first version) |
|---|---|
| • Weighted site/road graph; Kruskal + DSU; unavailable-road filtering<br>• Incremental three-road-failure repair<br>• Multi-station DP allocation with backtracking<br>• Brute-force check, edge cases, sensitivity analysis<br>• Synthetic dataset and web dashboard | • Choosing where to build stations (facility location): sites are given<br>• Physical chargers, payments, meters or grid control; OCPP<br>• Machine learning, blockchain<br>• Mobile app, real-time GPS, queues, prices, charging duration |

A **synthetic** urban network inspired by Karachi areas (DHA, Clifton, Saddar, PECHS, Gulshan, Gulistan-e-Johar, North Nazimabad, Korangi, Malir, Airport). All costs, capacities, severities and demands are synthetic and labeled as such.

## 04 Application Specifications and Constraints

| Specifications (what the application does) | Constraints (rules the application must obey) |
|---|---|
| **Dataset:** ≥10 locations (id, name, GPS, severity) incl. one hub; ≥20 roads (id, u, v, cost, available flag); 3–4 stations with distinct integer-unit capacities; ≥20 EV requests (id, origin, kWh, units, benefit).<br>**Validation:** reject duplicate IDs, negative costs/capacities, invalid endpoints.<br>**Scale:** networks up to 200 locations; 10–100 EV requests.<br>**Outputs:** selected edges, total cost, named components (incl. isolated sites and hub component); per-EV assignment or unserved reason; station loads; energy and benefit; runtimes. | **C1** Undirected graph, non-negative costs; unavailable roads never selected.<br>**C2** Each EV served at most once, by one station, with its full energy.<br>**C3** Station load never exceeds capacity.<br>**C4** Eligibility: only an active station in the EV's component.<br>**C5** 1 unit = 10 kWh; requests rounded up, capacities down.<br>**C6** Dense DP limited to k = 3–4 stations (states = $\prod(C_j+1)$).<br>**C7** Charging time, queues, prices, grid limits, safety risk are discussed, not optimized. |

## 05 Conflicting Requirements and Trade-offs

| Conflict | Trade-off chosen and justification |
|---|---|
| **Connection cost vs coverage** | Kruskal minimizes cost, not coverage; we report component count/membership and never claim maximum coverage. |
| **Safety vs low cost** | Unavailable roads are a hard filter: a cheaper unsafe road is never selected. |
| **Capacity vs demand; benefit vs fairness** | Maximize total benefit under hard capacity limits; some EVs stay unserved (reasons reported). Fairness not guaranteed; weights disclosed. |
| **Urgency / time-decay vs computation** | Optional decay $v_i(t) = v_i \cdot e^{-\lambda t}$, fixed per run so DP stays exact; off by default. |
| **Exactness vs scalability** | Exact DP with k = 3–4 stations and 10 kWh units; the greedy network scales to 200 sites. |

# 06 Methodology

## 6.1 Network planning: Kruskal's algorithm (Greedy)

G = (V, E), n = |V| sites (severity s(v), one hub), m = |E| roads (cost w(e) ≥ 0, availability flag). Unavailable roads are removed first; a disconnected graph yields a Minimum Spanning Forest, with every component listed by name and the hub's identified.

```
KRUSKAL-MSF(V, E):   # n = |V|, m = |E|
    A ← available edges of E
    sort A by (cost asc, s(u)+s(v) desc, min(u,v) asc, max(u,v) asc)
    MAKE-SET(v) for every v;   F ← empty
    for each (u, v, cost) in A:
        if FIND(u) != FIND(v):  add (u, v, cost) to F;  UNION(u, v)
    return F, components
```

Severity only breaks equal-cost ties (it never changes the minimum cost), so critical sites are connected earlier at no extra cost. **Correctness:** cut property (cheapest edge across a cut is safe for some MST), DSU rejects cycles, and optimal substructure holds on contracted components. **Complexity:** time O(n + m log m + m α(n)) = O(m log m), space O(n + m). **Why not Prim:** Kruskal works on the edge list, returns a forest without restarts, and its DSU supports component/eligibility checks and repair.

## 6.2 Incremental repair after three road failures

Three roads are marked unavailable. For each failed road in forest F: remove it, find the cut sides $C_1$, $C_2$, add the lowest-ranked surviving edge between them (same order as Kruskal), or keep the split if none exists. Failed roads outside F need no change. Checked against full Kruskal: edges available, acyclic, components spanned, equal total cost (edge sets may differ only on ties). Complexity O(r(n + m)) with r = 3, i.e. O(n + m); a simple repair, not a dynamic-MST structure, so speed-up is measured, not assumed.

## 6.3 Energy allocation: exact Dynamic Programming

q EV requests, k stations; EV i needs $w_i$ units, gives benefit $v_i$, has eligible set $E_i$ (stations in its component); station j has capacity $C_j$; $x_{ij} = 1$ if EV i is assigned to j.

**maximize** $\sum_i \sum_j v_i \cdot x_{ij}$ **subject to** $\sum_i w_i \cdot x_{ij} \le C_j$; $\sum_j x_{ij} \le 1$; $x_{ij} = 0$ if j is not in $E_i$; $x_{ij}$ in {0,1}

**Recurrence:** DP[i][$c_1 \ldots c_k$] = best benefit from the first i EVs with budgets ($c_1 \ldots c_k$); DP[0][·] = 0. Either skip (DP[i−1][c]) or assign to an eligible j with $w_i \le c_j$: $v_i$ + DP[i−1][c with $c_j - w_i$]; answer DP[q][$C_1 \ldots C_k$], with a decision table backtracked to recover the assignment. Ties prefer more EVs served, then lower station ID. Unserved EVs get a reason: no eligible station, too large for every eligible station, or lost to capacity competition.

**Why DP, not greedy or k independent knapsacks:** with 50 units, greedy picks EV A (50, benefit 100) = 100, but EV B + EV C (20 each, benefit 60 each) = 120; independent knapsacks could assign one EV to two stations. **Complexity:** with $S = \prod(C_j + 1)$, time O(q·k·S), space O(q·S) — pseudo-polynomial, exponential in k (hence C6). Brute force enumerates $(k+1)^q$ assignments, so it runs only for q ≤ 10.

# 07 Validation, Edge Cases and Experiments

Every run checks: no capacity exceeded, no EV assigned twice, every assignment eligible, forest acyclic and spanning, no unavailable edge selected, DP benefit equal to brute force (q ≤ 10). Runtimes use the same machine, language and input, repeated several times.

| Edge case | Expected behavior | Edge case | Expected behavior |
|---|---|---|---|
| Disconnected / isolated site | MSF; components and isolated sites listed by name | Unavailable edge | Never selected, before or after repair |
| Energy over-demand | Capacities respected; max-benefit subset; unserved IDs listed | Cycle among cheap edges | DSU rejects the closing edge |
| EV larger than every eligible station | Unserved, reason stated | Equal-cost edges | Deterministic tie-break; cost still minimal |
| Zero requests / zero capacity | Valid empty or partial allocation, no crash | Failed road in forest and one outside | Repair only where needed; cost equals full Kruskal |

**Experiments** (each reports edges, cost, components, EVs served/unserved, energy, benefit): **E1** baseline; **E2** three-road failure (incremental vs full Kruskal: validity, cost, edges, runtime); **E3** remove the two highest-severity locations and incident roads, deactivate stations there, rerun both stages; **E4** reduce station capacity by a stated percentage and increase EV demand (new unique IDs), rerun DP. Network scaling tests use up to 200 locations.

# 08 Timeline, Outcomes and Limitations

**Weeks 1:** Union-Find, Kruskal with tie-break and road filtering, incremental repair, DP, brute force, synthetic dataset. **2:** validation, edge-case tests, experiments E1–E4. **2–3:** FastAPI backend and PostgreSQL. **3-4:** dashboard, map, integration. **4:** final report.

**Expected outcomes:** a verified minimum-cost network (or forest), a repair matching full recomputation in cost, an allocation matching brute force, and runtimes consistent with theory. **Limitations:** synthetic data; MST minimizes connection cost only, not travel time, coverage or safety; benefit scores encode designer priorities and do not guarantee fairness; unit rounding can affect feasibility; indivisible jobs; k limited by the multidimensional DP. A DAA decision-support prototype, not a deployable charging-management system.